

ES6 Restaurant Menu Analyzer

Name: Amna Ata Ur Rehman

Registration Number: JUN26-FE25-23

Course: Web Foundations and JavaScript Mastery

Project title: ES6 Restaurant Menu Analyzer

Submission Date: 26 July 2026

Task Overview

Introduction

The objective of this mini project was to build a Restaurant Menu Analyzer using modern JavaScript ES6 features and array methods. The application stores restaurant menu data, performs different data analysis operations, and displays the results dynamically in the browser.

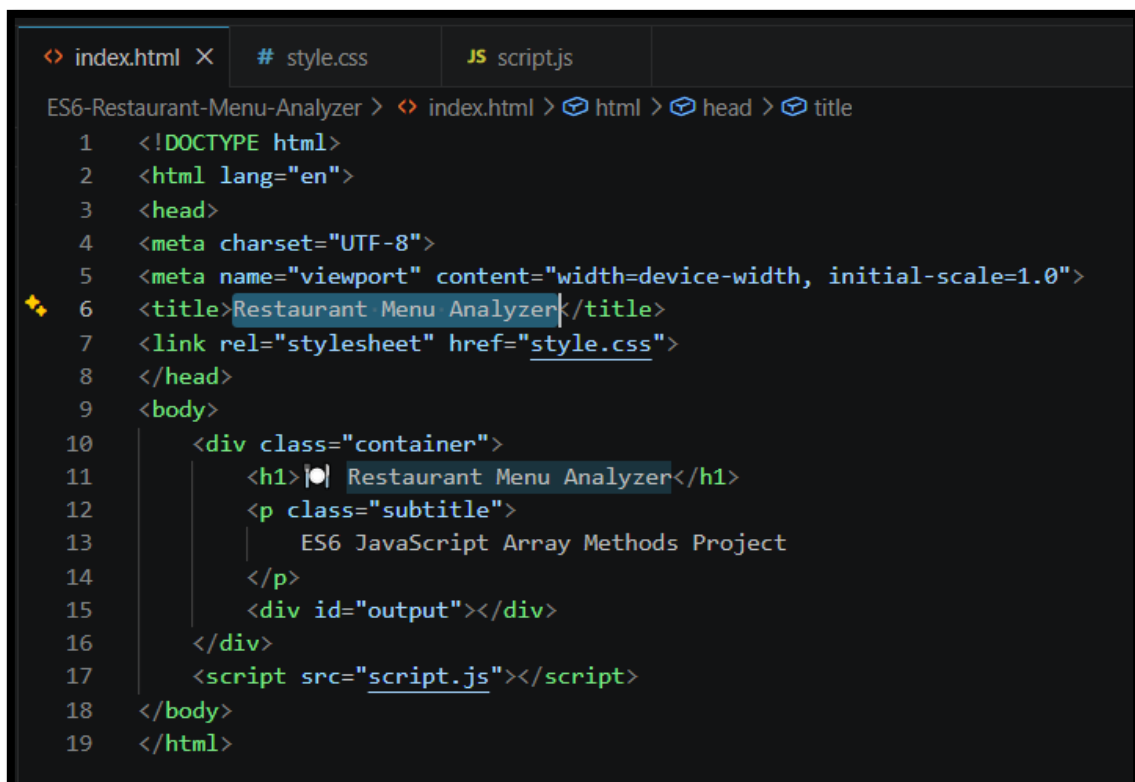
This project demonstrates practical implementation of ES6 concepts including destructuring, spread syntax, template literals, arrow functions, and various array methods.

Technologies Used

- HTML5
- CSS3
- JavaScript ES6
- VS Code
- Google Chrome

Screenshot # 01

- Basic HTML structure created for the Restaurant Menu Analyzer, including the output container used to render results.



The screenshot shows a VS Code editor with three tabs: index.html, style.css, and script.js. The index.html tab is active, displaying the following HTML code:

```
ES6-Restaurant-Menu-Analyzer > <> index.html > html > head > title
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  <title>Restaurant Menu Analyzer</title>
7  <link rel="stylesheet" href="style.css">
8  </head>
9  <body>
10     <div class="container">
11         <h1>Restaurant Menu Analyzer</h1>
12         <p class="subtitle">
13             ES6 JavaScript Array Methods Project
14         </p>
15         <div id="output"></div>
16     </div>
17     <script src="script.js"></script>
18 </body>
19 </html>
```

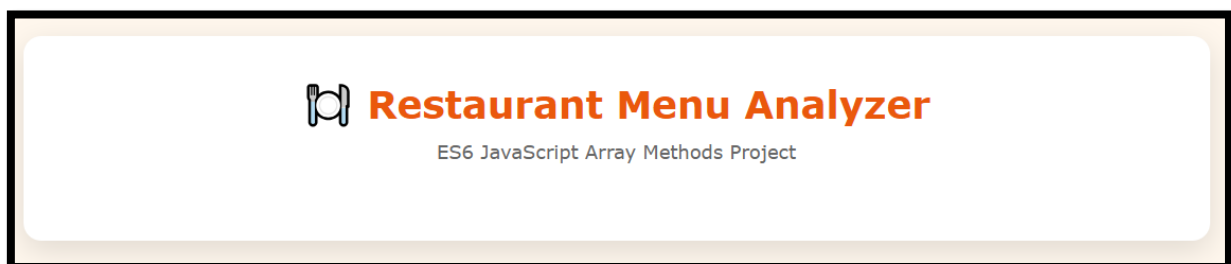
Screenshot # 2

- CSS styling used to design the layout and appearance of the user interface.

```
<> index.html  # style.css  X  JS script.js
ES6-Restaurant-Menu-Analyzer > # style.css > .card p
1  *{
2  margin:0;
3  padding:0;
4  box-sizing:border-box;
5  }
6  body{
7      font-family:Verdana,sans-serif;
8      background: #ffff7ed;
9      padding:40px;
10 }
11 .container{
12     max-width:1000px;
13     margin:auto;
14     background: #white;
15     padding:35px;
16     border-radius:15px;
17     box-shadow:0 8px 25px #rgba(0,0,0,.12);
18 }
19 h1{
20     text-align:center;
21     color: #ea580c;
22     margin-bottom:10px;
23 }
24 .subtitle{
25     text-align:center;
26     margin-bottom:30px;
27     color: #666;
28 }
29 .card{
30     background: #ffffbeb;
31     border-left:6px solid #f97316;
32     padding:20px;
33     margin-bottom:20px;
34     border-radius:10px;
```

Screenshot 3:

- Initial project interface displayed in the browser.

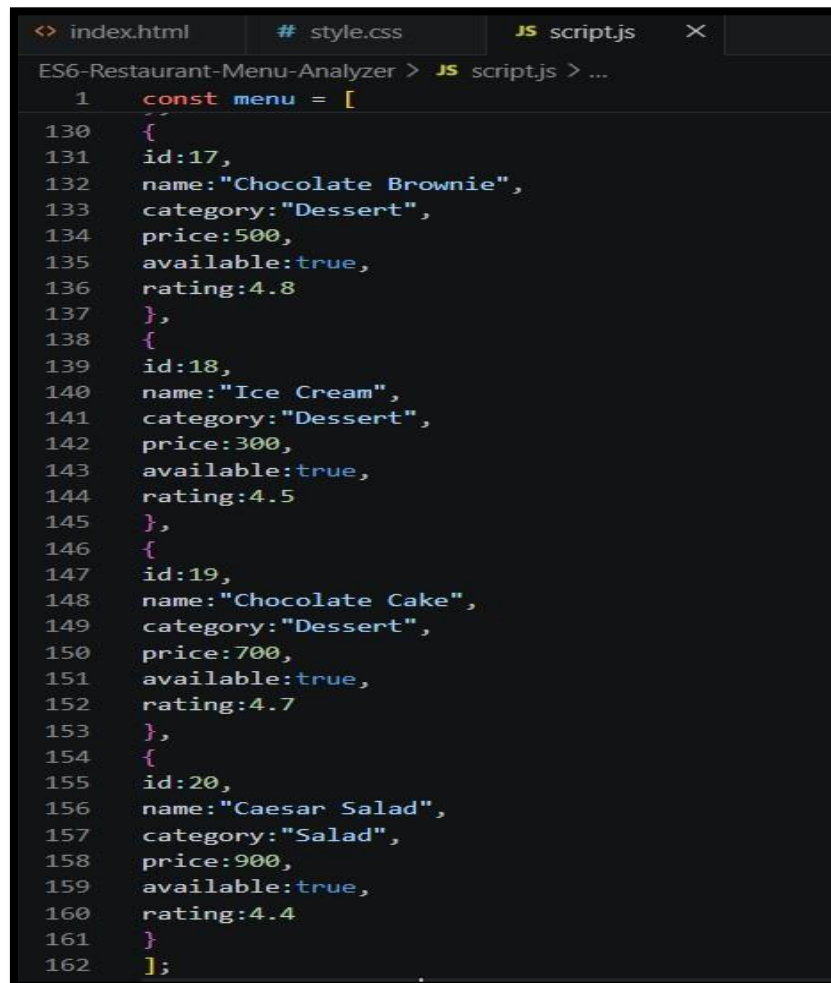


Restaurant Menu Data

Twenty restaurant menu items were created using JavaScript objects. Each object stores important information including item ID, menu name, category, price, availability, and customer rating.

Screenshot # 4

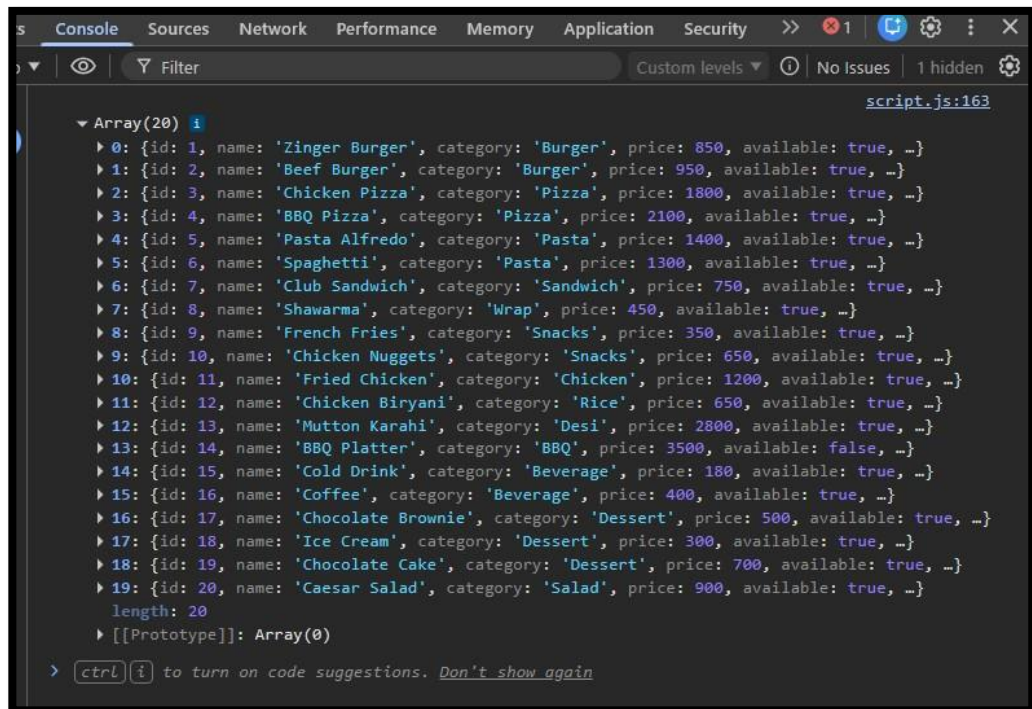
- Array containing twenty restaurant menu objects, each holding ID, name, category, price, availability, and rating.

A screenshot of a code editor with a dark theme. The editor has three tabs at the top: 'index.html', 'style.css', and 'script.js'. The 'script.js' tab is active. The code in the editor is as follows:

```
ES6-Restaurant-Menu-Analyzer > JS script.js > ...
1  const menu = [
130 {
131   id:17,
132   name:"Chocolate Brownie",
133   category:"Dessert",
134   price:500,
135   available:true,
136   rating:4.8
137 },
138 {
139   id:18,
140   name:"Ice Cream",
141   category:"Dessert",
142   price:300,
143   available:true,
144   rating:4.5
145 },
146 {
147   id:19,
148   name:"Chocolate Cake",
149   category:"Dessert",
150   price:700,
151   available:true,
152   rating:4.7
153 },
154 {
155   id:20,
156   name:"Caesar Salad",
157   category:"Salad",
158   price:900,
159   available:true,
160   rating:4.4
161 },
162 ];
```

Screenshot # 5

- Menu data logged to the console, confirming the array was created correctly.



ES6 Features

The project uses modern JavaScript ES6 features to write cleaner and more efficient code.

Implemented Features:

- Object Destructuring
- Spread Operator
- Arrow Functions
- Template Literals

Object destructuring was used to extract individual menu item details such as name, price, and category directly from a menu object. The spread operator was then used to create a new menu object based on an existing item, extending it with additional fields such as a discount and a chef's special flag.

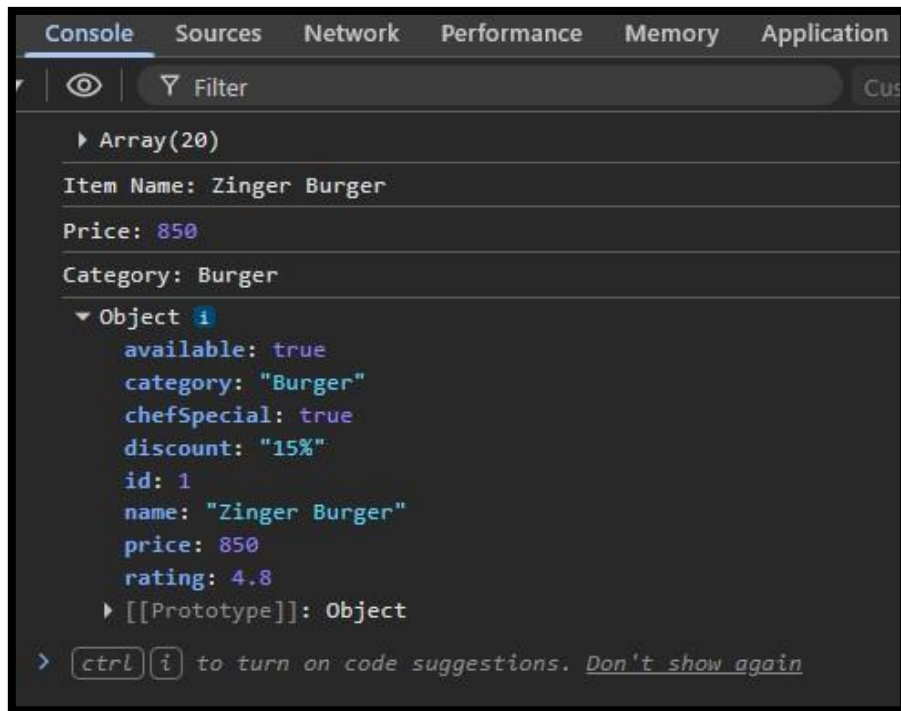
Screenshot # 6

- Object destructuring used to extract menu item details, and the spread operator used to build a new menu object.

```
165 // ===== Destructuring =====
166 const { name, price, category } = menu[0];
167 console.log("Item Name:", name);
168 console.log("Price:", price);
169 console.log("Category:", category);
170
171 // ===== Spread Operator =====
172 const specialMenu = {
173   ...menu[0],
174   discount: "15%",
175   chefSpecial: true
176 };
177 console.log(specialMenu);
178
```

Screenshot # 7

- Console output showing the destructured values and the new object produced with the spread operator.



Array Methods Implementation

Several ES6 array methods were used to analyze the restaurant menu data.

map() — Used to create a new array containing only the menu item names.

filter() — Used to display premium menu items, defined as items priced above Rs. 1000.

```
183 // ===== filter() =====
184 const premiumItems = menu.filter(item => item.price > 1000);
185 console.log(premiumItems);
186
```

find() — Used to search for a specific menu item by name.

```
// ===== find() =====
const searchedItem = menu.find(item => item.name === "Chicken Pizza");
console.log(searchedItem);
```

Screenshot # 10

➤ Console output of the find() method, showing the matched menu item.

```
script.js:189
▼ {id: 3, name: 'Chicken Pizza', category: 'Pizza', price: 1800, available: true, ...} ⓘ
  available: true
  category: "Pizza"
  id: 3
  name: "Chicken Pizza"
  price: 1800
  rating: 4.9
  ▶ [[Prototype]]: Object
```

some() — Used to check whether any menu item is currently unavailable.

```
// ===== some() =====
const unavailableItem = menu.some(item => item.available === false);
console.log(unavailableItem);
```

Screenshot # 12

➤ Console output of the some() method.

```
true
> [ctrl] ⓘ to turn on code suggestions. Don't show again
```

every() — Used to verify whether all menu items have a rating of 4 or above.

reduce() — Used to calculate the total combined price of all menu items.

Screenshot # 13

➤ CODE for reduce() method, every(), Summary object

```

195 // ===== reduce() =====
196 const totalMenuPrice = menu.reduce((total, item) => total + item.price, 0);
197 console.log(totalMenuPrice);
198
199 // ===== every() =====
200 const goodRatings = menu.every(item => item.rating >= 4);
201 console.log(goodRatings);
202
203 // ===== Summary Object =====
204 const summary = {
205   totalItems: menu.length,
206   totalMenuPrice: totalMenuPrice,
207   averageRating:
208     (menu.reduce((sum, item) => sum + item.rating, 0) / menu.length).toFixed(1),
209   highestPrice:
210     Math.max(...menu.map(item => item.price)),
211   lowestPrice:
212     Math.min(...menu.map(item => item.price))
213 };
214 console.log(summary);
215

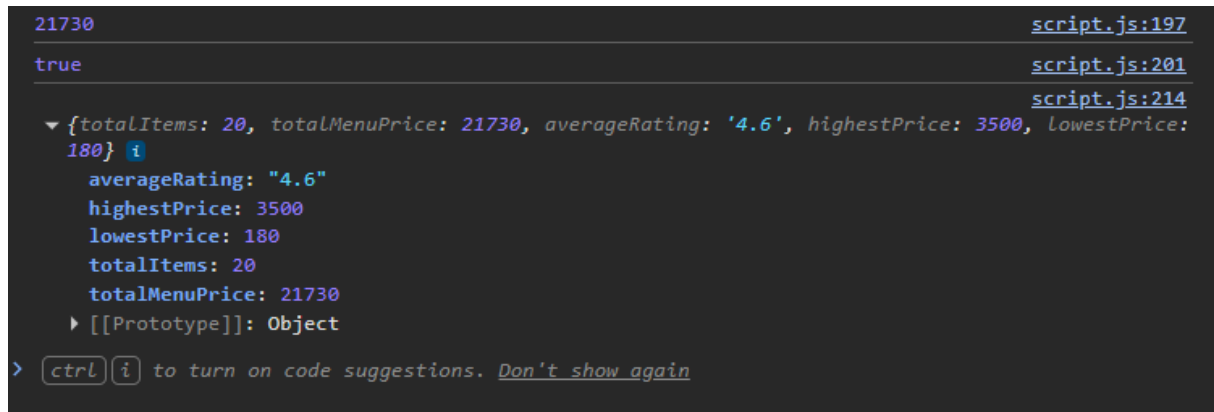
```

The map(), every(), and reduce() methods were implemented and verified through the browser console in the same way, and are included directly in the project's JavaScript file.

A summary object was created to display important restaurant statistics such as the total number of menu items, the total menu price, the average rating, the highest-priced item, and the lowest-priced item.

Screenshot # 14

- Console Output of the `reduce()` method, `every()`, Summary object



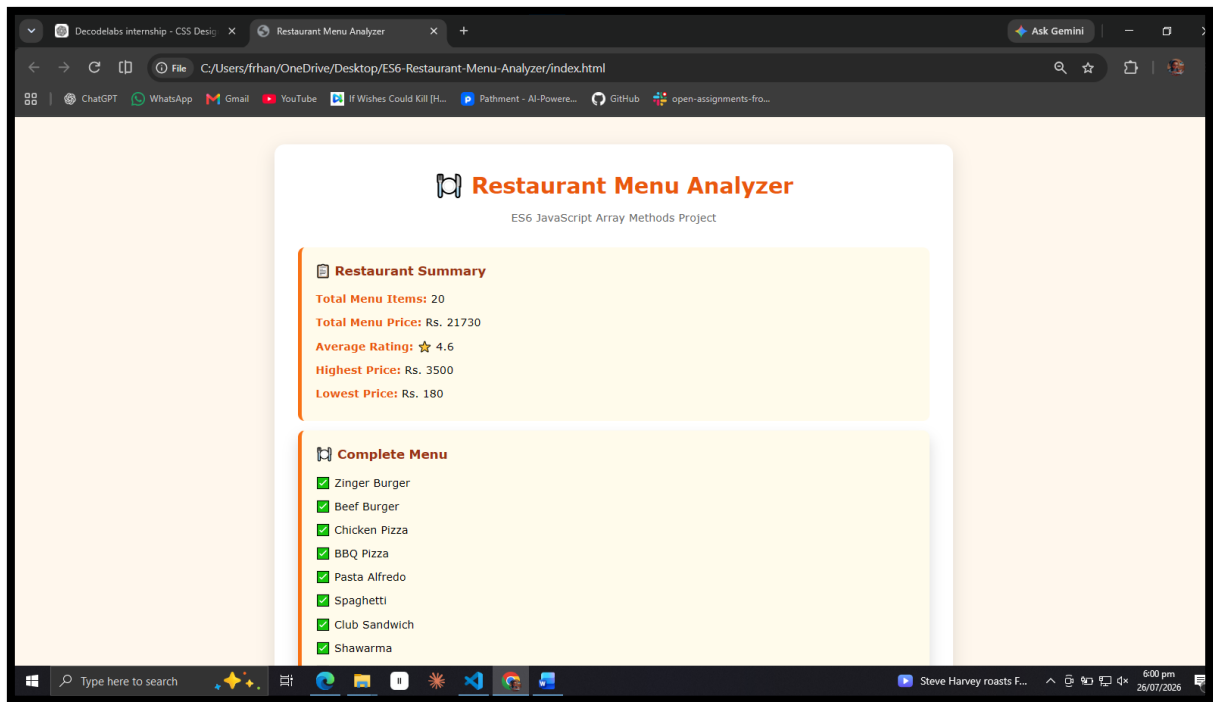
```
21730 script.js:197
true script.js:201
▼ {totalItems: 20, totalMenuPrice: 21730, averageRating: '4.6', highestPrice: 3500, lowestPrice: 180} i script.js:214
  averageRating: "4.6"
  highestPrice: 3500
  lowestPrice: 180
  totalItems: 20
  totalMenuPrice: 21730
  ▶ [[Prototype]]: Object
> [ctrl][i] to turn on code suggestions. Don't show again
```

Final Output

All analyzed information is displayed dynamically inside the browser using JavaScript DOM manipulation. The final output includes a restaurant summary card, the complete list of menu items, the list of premium items, the search result for a specific item, and an availability check confirming whether any item is out of stock and whether all items meet the minimum rating requirement.

Screenshot # 14

- Final output of Restaurant Menu Analyzer.



Conclusion

This project improved my practical understanding of JavaScript ES6 concepts, object manipulation, array methods, and DOM manipulation. I learned how to organize and analyze restaurant menu data efficiently while creating a clean and user-friendly interface.